

Funciones de la red neuronal y proceso de la implementación

1.1 Funciones para el preprocesamiento de audio e imágenes

A continuación, se presentarán las funciones definidas en el código de Colab encargadas de realizar el cargue y preprocesamiento de audio, así como la conversión y creación de imágenes.

Función load_audio. Se encarga de cargar los archivos de audio .wav y convertirlos a un formato capaz de ser usado por TensorFlow.

Lo hace re muestreando el audio a una frecuencia de muestreo de 16 kHz y convirtiéndolo a un formato de punto flotante de 32 bits. Finalmente, devuelve la forma de onda y la frecuencia de muestreo.

Función load_audio_files. Es la encargada de etiquetar los audios de la base de datos en la red neuronal. Primero recorre todos los archivos .wav del directorio para extraerles la información del nombre del archivo previamente etiquetado. Esta información es el comando, nivel o entorno de ruido y el ID del hablante o persona.

Luego verifica si el comando coincide con las etiquetas proporcionadas (labels) y llama a la función anteriormente definida (load_audio) para agregarles la forma de onda, frecuencia de muestreo, comando, ruido e ID. Para finalmente devolver una lista con estos 5 datos del audio.

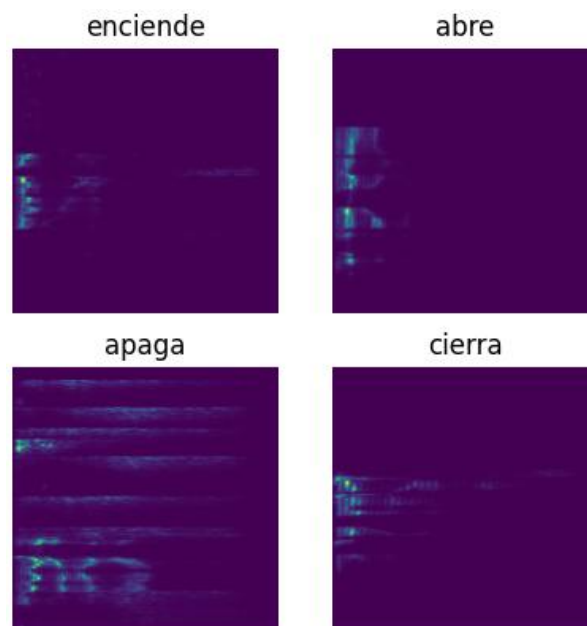
Función get_spectrogram. Esta función toma una señal de audio (waveform) y la convierte en un espectrograma. Primero, verifica si la señal tiene menos de 8000 muestras y, si es necesario, la rellena con ceros para asegurar que todas las entradas tengan la misma longitud. Luego, convierte la señal en un espectrograma utilizando la Transformada de Fourier de Tiempo Corto (STFT), que divide la señal en segmentos de 256 muestras con un paso de 128

muestras y calcula su contenido frecuencial en cada segmento. Para finalizar obtiene la magnitud del espectrograma para eliminar la información de fase y devolver la representación espectral de la señal de audio.

Función `plot_spectrogram`. Como su nombre lo dice, visualiza los espectrogramas anteriormente creados. En la Figura 1 se observan 4 imágenes de espectrogramas, en escala de grises, del conjunto de datos de entrenamiento, junto a sus etiquetas.

Figura 1

Imágenes de algunos espectrogramas del conjunto de entrenamiento



Función `create_images`. Para finalizar esta etapa de la red neuronal, la función `create_images` crea directorios para almacenar las imágenes de entrenamiento y prueba, luego recorre el conjunto de datos de audio llamando a la función `get_spectrogram` para calcular el espectrograma de cada forma de onda y guardar la imagen en cada directorio correspondiente.

1.2 Procesos de la implementación

1.2.1 Proceso de grabar un audio

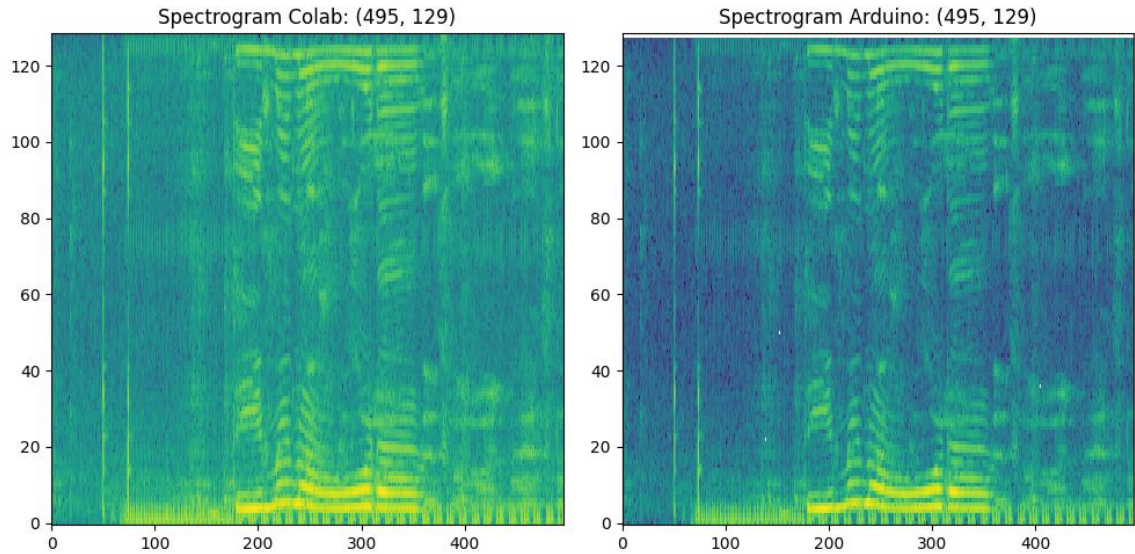
El proceso de grabación comienza al presionar el botón 3 y se detiene después de 2.5 segundos. Primero, se verifica que la tarjeta SD esté correctamente inicializada. Si el archivo destinado para la grabación ya existe, se elimina para evitar conflictos. A continuación, se abre un nuevo archivo en la tarjeta SD donde se almacenan los datos de audio. Durante la grabación, los datos capturados del micrófono se procesan y se escriben continuamente en el archivo gracias a la función `audioCopier.copy()`. Al detener la grabación, el archivo se cierra y los datos se guardan de manera segura.

1.2.2 Proceso de analizar la señal de audio

Se empieza inicializando las variables, tales como el número de canales, el tamaño de la ventana, el paso de las ventanas y la frecuencia de muestreo, así como en Colab. Luego, se verifica que la longitud del audio sea de mínimo 8000 muestras y se hace el proceso de inicialización, procesado y redimensionamiento del espectrograma, el cual se muestrea mediante las ventanas de Hann y se procesa haciendo uso de la Transformada Directa de Fourier (DFT). Este espectrograma se guarda en la PSRAM integrada en la placa de desarrollo. A su vez, se genera un archivo csv para analizar el espectrograma en Colab. En la Figura 2 se visualiza la comparación entre espectrogramas de uno de los audios del dataset.

Figura 2

Comparación de espectrogramas



1.2.3 Proceso de cargar la red neuronal en el microcontrolador

Como se mencionó anteriormente, ya está creado el archivo llamado `mejor_modelo.h` el cual contiene al mejor modelo creado en Colab. Luego se ejecuta un modelo de aprendizaje automático con TensorFlow Lite. Se define un búfer de memoria (`tensor_arena`) de 40 KB definido en la PSRAM para alojar los tensores del modelo. Para acabar, se define un puntero a un `MicroInterpreter`, que será el encargado de ejecutar el modelo en el microcontrolador, aunque aún no se ha inicializado.

1.2.4 Proceso de clasificar este nuevo audio en la red

Se toma el espectrograma redimensionado y se cuantiza como tipo entero de 8 bits, luego se ingresa al modelo para hacer la respectiva inferencia para la salida, esta se convierte también en tipo entero de 8 bits. Finalmente, se asignan valores de 0 a 1 a cada una de las clases para su respectiva clasificación. Donde el comando con mayor puntaje es el predicho.

1.2.5 Proceso de salida de audio

Después de clasificar el nuevo audio y obtener el comando con mayor probabilidad se envía la etiqueta de la predicción seleccionada a un if el cual elige uno de 6 audios pregrabados con ayuda de la página narakeet (Apéndice B) y se reproduce el archivo etiqueta.wav. Donde etiqueta varía según el comando predicho. Esto nos permite demostrar la predicción del sistema de forma auditiva.